

MLCC 2026
Train and Test and all that
or
An introduction to statistical machine learning

Lorenzo Rosasco
MaLGa, DIBRIS, UNIGE;
IIT

About this class

1. Introduce the data model in statistical learning.
2. Introduce training ...
3. and testing.

Outline

Statistical learning

Training

Testing

Data

$$(x_1, y_1), \dots, (x_n, y_n)$$

Basic setting: $x \in \mathcal{X} = \mathbb{R}^d$, $y \in \mathcal{Y} = \mathbb{R}$ or $\mathcal{Y} = \{\pm 1\}$.

Many more settings: graphs, histograms, strings...

Where are the data coming from?

$$(x_1, y_1), \dots, (x_n, y_n)$$

$$(x_{\text{new}}, y_{\text{new}})$$

Statistical machine learning

$$(x_i, y_i) \sim P(x, y), \quad i = 1, \dots, n$$

$$(x_{\text{new}}, y_{\text{new}}) \sim P(x, y)$$

Why probabilities?

- ▶ Data generating process.
- ▶ Epistemic uncertainty: noise, finite sampling¹
- ▶ Aleatoric uncertainty: data are inherently variable.²

¹But also missing variables, poor instrumentation, resolution, or an unknown systematic effect.

²Think: same patient, same measurements, different outcome.

I.I.D.

$$(x_1, y_1), \dots, (x_n, y_n) \sim P^n(x, y)$$

I.I.D. = independent and identically distributed.

It is a very strong assumption.

“All models are wrong, but some are useful.”

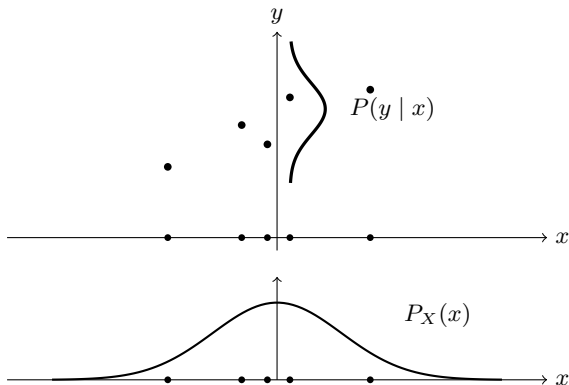
George Box

Conditional and marginal

$$P(x, y) = P(y \mid x)P_X(x)$$

Conditional and marginal in regression

$$P(x, y) = P(y | x)P_X(x)$$



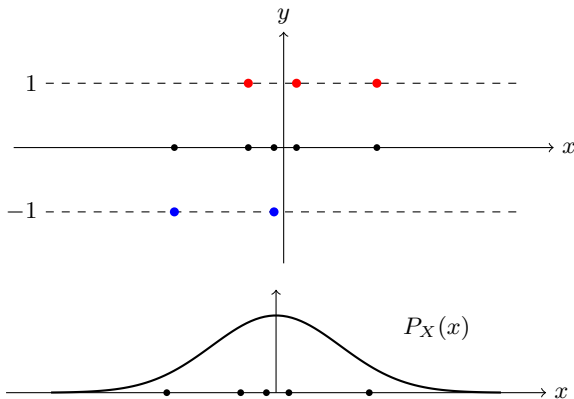
Regression model: $i = 1, \dots, n$

$$y_i = f_{\star}(x_i) + \epsilon_i$$

$$x_i \sim P_X, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2) \quad \Rightarrow \quad P(y_i | x_i) = \mathcal{N}(f_{\star}(x_i), \sigma^2).$$

Conditional and marginal in classification

$$P(x, y) = P(y | x)P_X(x)$$



Classification model: $i = 1, \dots, n$

$$y_i \in \{-1, 1\}, \quad \mathbb{P}(y_i = 1 | x_i) = f_\star(x_i)$$

$$x_i \sim P_X, \quad \mathbb{P}(y_i = -1 | x_i) = 1 - f_\star(x_i).$$

Prediction functions

$$P(y \mid x)$$

$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

Loss function

$$\ell(y, f(x))$$

Loss function

$$\ell(y, f(x))$$

Regression: $\ell(y - f(x))$

square

$$(y - f(x))^2$$

absolute value

$$|y - f(x)|$$

Loss function

$$\ell(y, f(x))$$

Regression: $\ell(y - f(x))$

square $(y - f(x))^2$

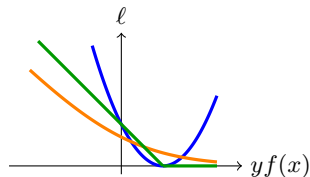
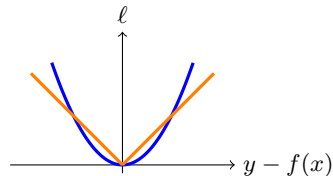
absolute value $|y - f(x)|$

Classification: $\ell(yf(x))$

square $(1 - yf(x))^2$

logistic $\log(1 + e^{-yf(x)})$

hinge $|1 - yf(x)|_+$



Test error

The goal of learning is to predict well new data.

$$(x_{\text{new}}, y_{\text{new}}).$$

Test error= error on new data,

$$R_{\text{Test}}(f) = \frac{1}{N_{\text{new}}} \sum_{(x_{\text{new}}, y_{\text{new}})} \ell(y_{\text{new}}, f(x_{\text{new}})).$$

Test error or expected risk?

$$R(f) = \mathbb{E}_{(x,y) \sim P} [\ell(y, f(x))]$$

Writing the expectation explicitly,

$$R(f) = \int_{\mathcal{X} \times \mathcal{Y}} \ell(y, f(x)) dP(x, y).$$

Interpretation: test error on infinite data!

A weighted *sum* of the errors, with higher weights for data that happens more often

The learning problem

Given data $(x_1, y_1), \dots, (x_n, y_n)$ find a prediction function $\hat{f}$ making the expected (test) risk

$$R(f) = \mathbb{E}_{(x,y) \sim P} [\ell(y, f(x))]$$

small.

But we cannot compute R , so what?

Outline

Statistical learning

Training

Testing

Basic idea

Given data $(x_1, y_1), \dots, (x_n, y_n)$ let

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i)).$$

Intuition

$$\hat{R}(f) \approx R(f).$$

Empirical mean and mean

For a **fixed** f , let $Z = \ell(y, f(x))$ and $Z_i = \ell(y_i, f(x_i))$.

Then

$$R(f) = \mathbb{E}[Z] = \mu_Z$$

and

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^n Z_i = \hat{\mu}_Z.$$

It is a basic fact that $\mathbb{E}[\hat{\mu}_Z] = \mu_Z$, that is

$$\mathbb{E}[\hat{R}(f)] = R(f).$$

Empirical risk minimization (ERM)

$$\widehat{R}(f) \approx R(f).$$

A basic idea

$$\min_{f \in \mathcal{H}} \widehat{R}(f) \approx \min R(f)$$

Representation

$$\min_{f \in \mathcal{H}} \hat{R}(f)$$

What is $\mathcal{H}$?

$\mathcal{H}$ is the hypothesis space (where we search for solutions).

Choosing $\mathcal{H}$ = representation/parametrization of the functions.

$$\underbrace{w^\top x}_{\text{linear}}$$

$$\underbrace{w^\top \Phi(x)}_{\text{nonlinear}}$$

$$\underbrace{w^\top \sigma(W^T x)}_{\text{nonlinear}}$$

Representation (cont.)

$$\mathcal{H} = \{f_\theta \mid \theta \in \Theta\}.$$

Here Θ is the weight/parameter space.

We can rewrite ERM as

$$\min_{\theta \in \Theta} \hat{R}(\theta), \quad \hat{R}(\theta) = \hat{R}(f_\theta) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f_\theta(x_i))$$

Optimization

$$\min_{\theta \in \Theta} \widehat{R}(\theta), \quad \widehat{R}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f_{\theta}(x_i)).$$

Gradient descent

$$\theta_{t+1} = \theta_t - \eta_t \nabla \widehat{R}(\theta_t)$$

Stochastic gradient descent

$$\theta_{t+1} = \theta_t - \eta_t \nabla \ell(y_t, f_{\theta_t}(x_t))$$

ERM recipe

$$\min_{\theta \in \Theta} \hat{R}(\theta)$$

- ▶ Empirical approximation.
- ▶ Representation.
- ▶ Optimization

Most of the week is about this!

But not all algorithms are ERM!

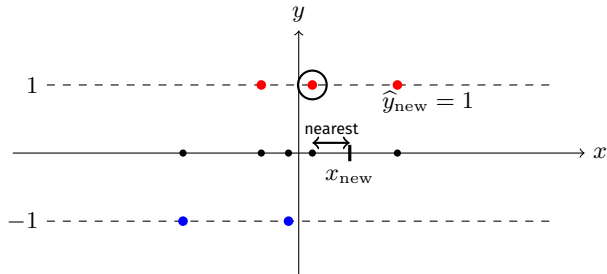
Local methods: nearest neighbor

Idea: nearby inputs have same outputs.

Nearest neighbor Given $(x_i, y_i)_{i=1}^n$, predict a new point x_{new} label using the e closest point label,

$$\hat{y}_{\text{new}} = y_{i_*}, \quad i_* \in \arg \min_{i=1, \dots, n} \|x_{\text{new}} - x_i\|.$$

Nearest neighbor illustrated



K-nearest neighbor

Idea: average over the K closest points.

Let $\hat{N}_K(x_{\text{new}})$ be the indices of the K closest data points to x_{new} .

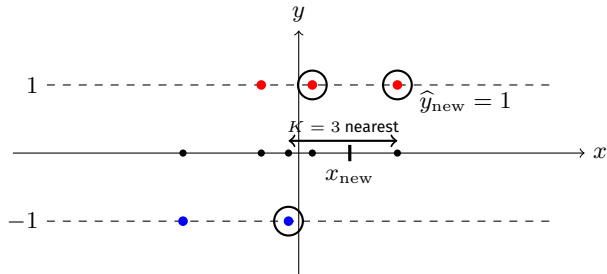
For classification,

$$\hat{f}(x_{\text{new}}) = \text{majority}\{y_i \mid i \in \hat{N}_K(x_{\text{new}})\}.$$

For regression,

$$\hat{f}(x_{\text{new}}) = \frac{1}{K} \sum_{i \in \hat{N}_K(x_{\text{new}})} y_i.$$

K-nearest neighbor illustrated



Outline

Statistical learning

Training

Testing

Model assessment and selection

Different approaches, representation, optimization, or parameter settings give different solutions.

How can we pick the best?

Model assessment and selection (cont.)

Each of M algorithms returns an empirical function $\hat{f}_j$ computed on data $(x_1, y_1), \dots, (x_n, y_n)$.

Given

$$\hat{f}_1, \dots, \hat{f}_M$$

we ideally would like to compute

$$R(\hat{f}_j), \qquad R(\hat{f}_j) = \mathbb{E}[\ell(y, f(x))]$$

and select the solution with smallest expected risk ...

...but we only have data!

Assessing a model from data

We know that for each fixed f

$$\widehat{R}(f) \approx R(f), \quad \text{that is} \quad \mathbb{E}[\widehat{R}(f)] = R(f).$$

Given

$$\widehat{f}_1, \dots, \widehat{f}_M$$

can we just compute

$$\widehat{R}(\widehat{f}_j), \quad \widehat{R}(\widehat{f}_j) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i))]$$

and select the solution with smallest empirical risk?³

³Spoiler: NO. Why?)

Why not?

A bit of notation: $\hat{f}$ means $\hat{f}_{(x_1, y_1), \dots, (x_n, y_n)}$.

$\hat{f}$ is not a fixed function...it's a random variable!

In particular

$$\mathbb{E}[\ell(y_i, \hat{f}(x_i))] \neq R(\hat{f})$$

and

$$\mathbb{E}[\hat{R}(f)] = \frac{1}{n} \sum_{i=1}^n \mathbb{E}[\ell(y_i, \hat{f}(x_i))] \neq R(\hat{f})$$

The hold-out method

Split the data:

$$\{(x_1, y_1), \dots, (x_{\frac{n}{2}}, y_{\frac{n}{2}})\} \cup \{(x_{\frac{n}{2}+1}, y_{\frac{n}{2}+1}), \dots, (x_n, y_n)\}.$$

Use the first half (training) to compute

$$\hat{f}_1, \dots, \hat{f}_M.$$

Use the second half (test) to compute

$$\hat{R}_{\text{test}}(\hat{f}_j) = \frac{2}{n} \sum_{i=\frac{n}{2}+1}^n \ell(y_i, \hat{f}_j(x_i)).$$

Why hold-out works

$\hat{f}$ is computed on the training data, that is $\hat{f}_{(x_1, y_1), \dots, (x_{\frac{n}{2}}, y_{\frac{n}{2}})}$, but the test data are independent of $\hat{f}$!

For every test point (x_i, y_i) ,

$$\mathbb{E}_{(x_i, y_i)} \left[\ell(y_i, \hat{f}(x_i)) \right] = R(\hat{f}).$$

Hence

$$\mathbb{E}_{\text{test}} \left[\hat{R}_{\text{test}}(\hat{f}) \right] = R(\hat{f}).$$

The test error is unbiased for the risk of the trained model!

Training and testing

Training error:

$$\hat{R}_{\text{train}}(\hat{f}) = \frac{2}{n} \sum_{i=1}^{\frac{n}{2}} \ell(y_i, \hat{f}(x_i)).$$

Test error:

$$\hat{R}_{\text{test}}(\hat{f}) = \frac{2}{n} \sum_{i=\frac{n}{2}+1}^n \ell(y_i, \hat{f}(x_i)).$$

Training error measures fit to the data used to compute $\hat{f}$.

Test error estimates prediction on *new* data.

Cross validation

The hold-out method is based on one split.

Cross-validation applying the same idea using several (say V) splits, then averaging the test errors,

$$\hat{R}_{CV} = \frac{1}{V} \sum_{k=1}^V \hat{R}_{\text{test}}^{(V)}.$$

Flavors of cross validation

Hold-out. Split once into training and test data.

Repeated hold-out. Repeat random training/test splits and average the test errors.

V -fold cross-validation. Split the data into V blocks. Each block is used once as test data.

Leave-one-out cross-validation. V -fold with $V = n$, that is we test one point at the time.

Wrapping up

In this class, we discussed two basic processes in machine learning, namely training and testing. We justified and define the problem introducing the framework of statistical machine learning

Next class

Back to ERM considering basic linear models.